Check for updates

Co-Design of the Morphology and Actuation of Soft Robots for Locomotion

Merel van Diepen
ETH Zurich, EDAC, CLA F,
Tannenstrasse 3,
Zurich 8092, Switzerland

Kristina Shea[1]
Engineering Design and Computing
Laboratory CLA F,
Tannenstrasse 3,
Zurich 8092, Switzerland
e-mail: kshea@ethz.ch

In recent years, the field of soft robotics has received considerable attention due to its potential in increasing the safety of human-robot interaction. The design of soft robots possesses great challenges. For example, the longstanding challenge of co-design morphology and actuation makes designing them by hand a trial-and-error process. Earlier work presented by the authors proposes a computational design synthesis (CDS) method for the automated design of virtual, soft locomotion robot morphologies. This work extends the CDS method for morphologies with the automated co-design of actuation. Two methods are considered. In the first method, the actuation of designs is described by parametric actuation curves (PACs) that model feedforward actuation patterns. For every morphology in the design process, a set of PACs is optimized that assumes symmetric and cyclic gaits. The second method, soft actor-critic (SAC) reinforcement learning, removes this assumption as well as models feedback control for comparison. Adding PAC optimization to the CDS method is shown to improve the performance of the resulting designs and to achieve better results within less design iterations. SAC is, however, deemed less effective, due to the need for design specific problem tuning for each new morphology. The SAC experiments also show that the best found soft robot gaits are symmetric and cyclic, although this is not a constraint in the SAC problem formulation, thus verifying the assumptions made in the PAC formulation. To validate the search space modeled in the co-design CDS method, a state-of-the-art soft robot is replicated and compared. [DOI: 10.1115/1.4054522]

Keywords: computer-aided design, conceptual design, soft robotics, design automation, generative design, shape grammars

1 Introduction

There is an increasing interest in the design of soft robots since they have several advantages over rigid robots, e.g., increasing the safety of human-robot interaction [1] and achieving compliant behavior through the use of compliant materials, as opposed to sensors and computation [2,3]. In addition, soft robots allow for near free-form design. Despite the potential variety of soft robot designs, a majority of the current soft robot designs is based on a small set of bio-inspired morphologies, and there is usually only weak justification for the biological analogy used. Soft robot design without looking at nature is challenging, and the design of soft robots is not well understood yet in comparison to the design of rigid robots. The co-design of morphology and actuation is coupled, and the dynamics of the elastic bodies are often hard to predict, making designing original soft robot designs by hand a trial-and-error process that is time-consuming [4]. Design automation enables the evaluation of many design concepts and is not restricted to biological inspiration or the design bias of a human designer and can therefore help to explore the design space of soft robots.

A computational design synthesis (CDS) method to explore the design space of virtual and soft robots for locomotion is presented by van Diepen and Shea [5]. The CDS method automatically generates locomotive designs using a spatial grammar and simulates them using a rigid ball-spring model. The morphologies are evaluated based on the distance traveled in simulation and optimized using simulated annealing (SA), a global, stochastic optimization method. During the evaluation step, each actuator is actuated with one of six predefined actuation patterns, describing the target actuation at every time-step. By optimizing the morphology only, the design problem is simplified. However, it is difficult to deduce what the implications of this limitation on the design space are, i.e., which designs are excluded through the choice of predefined actuation patterns, and whether this constraint makes the design task more or less difficult.

In this article, the co-design of morphology and actuation of soft robots are presented to remove this restriction on the actuation. Two methods for the optimization of actuation are introduced. The first parametrically optimizes predefined actuation patterns. The actuation patterns are formulated as parametric actuation curves (PAC) and each PAC describes the actuation of one actuator. The set of PACs for each design is optimized with SA. PACs rely on the assumption that the gait is cyclic and a form of feedforward control. As a consequence, the solution space is limited and the resulting actuation patterns are sensitive to model inaccuracies. Thus, an actuation optimization method that is less constrained and results in feedback control is also investigated for comparison. The actuation problem is treated as an unsupervised learning problem and solved with a soft actor-critic (SAC) reinforcement learning (RL) method.

First, both PAC optimization and SAC are examined for morphologies generated with a CDS method for the design of morphology only and the performance is compared to predefined actuation. Based on the results, PAC optimization is integrated into the CDS method. As shown in Fig. 1, the co-design is weakly coupled; the morphology and actuation are altered in separate stages. Since morphology designs from consecutive iterations may differ greatly and can have a different number of actuators, a new actuation optimization is performed on every iteration. The resulting CDS method for

[1]Corresponding author.

Contributed by the Mechanisms and Robotics Committee of ASME for publication in the Journal of Mechanical Design. Manuscript received March 14, 2022; final manuscript received April 21, 2022; published online June 13, 2022. Assoc. Editor: Shikui Chen.

 Copyright © 2022 by ASME

from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAYCjq32Vp15yq9qlCmGqWPlWarSqf_Zsos6Btdcv1TskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

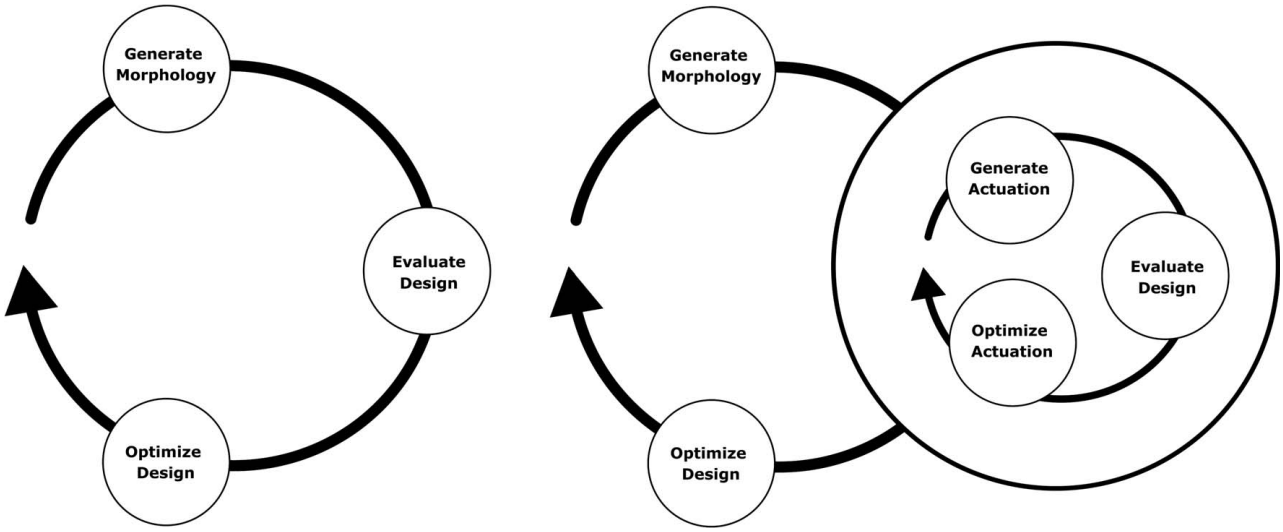


Fig. 1 An overview of CDS with (*a*) predefined actuation patterns and (*b*) with embedded co-design of morphology and actuation

the design of morphology and actuation, i.e., co-design is compared to the design of morphology only. This comparison shows that co-design achieves better performance with fewer iterations.

The key contributions of this article are as follows:

- A method for the optimization of the actuation patterns within a CDS method is presented, called PAC, and is shown to balance computational cost and performance, while not needing design specific tuning and thus is suitable for synthesis.
- The co-design of morphology and actuation results in designs with higher performance and larger variety than the designs resulting from the automated design of morphology with fixed actuation patterns.

The article is structured as follows. The relevant background is discussed in Sec. 2, followed by a methods section introducing the methods used for the design of morphologies, the optimization of actuation, and the co-design of morphology and actuation. In Sec. 4, the results of the experiments with the different methods are presented. In addition, the replication of a state-of-the-art soft robot is presented to assess the capabilities of the CDS method. Section 5 discusses the results, and the article is concluded in Sec. 6.

2 Background

The related work on the CDS of virtual robots can be separated into two classes of approaches; approaches that treat actuation as an optimization constraint, i.e., use a predefined actuation pattern and approaches that optimize actuation for each generated morphological design. The work on soft robot design automation previously published by van Diepen and Shea [5] belongs to the first class. The generated morphologies consist of actuators that bend according to predefined actuation patterns. Similarly, Cheney et al. [6] use predefined actuation patterns to expand and contract the voxels of soft material that make up their designs. The second class of approaches, the co-design of morphology and actuation, is first introduced by Sims [7], one of the first approaches to rigid robot design automation. In his work, directed graphs consisting of morphological nodes are used to generate designs. These morphological nodes contain nested graphs of neural nodes with a large range of different nodal functions. The behavior of the designs emerges from the combination of the nested network components. This neural network is optimized in the same way the morphology is; by mutation and crossover on the directed graph by a genetic algorithm. Similarly, Lipson and Pollack [8] use small neural networks embedded in the building blocks of the robot morphology to control evolved truss robots. Lund et al. [9] evolve rigid robot hardware with a genetic algorithm and simultaneously evolve the control using genetic programming. Moving away from robots to a different co-design problem, the control of the surfaces of the sailboat designs presented by Stump et al. [10] is learned using the proximal policy optimization implementation of Unity's ML Agent Toolkit. However, little information is given about the tuning of the parameters, and the effect of the RL method on the performance of the designs.

In conventional (rigid) robotics, controllers are often designed with the help of an explicit model of the system dynamics. However, due to the highly elastic material in soft robots and the challenges of modeling (self) collision, the design of soft robot controllers often relies on simulation-based optimization. To limit the size of the resulting optimization problem, assumptions on the desired gait are exploited; as demonstrated by the following two examples. Ishige et al. [11] control a caterpillar-like soft robot with a central pattern generator (CPG) and optimize the controller parameters using reinforcement learning. The size and architecture of the CPG are predefined and not subjected to optimization. The shape memory alloy muscles of a caterpillar-like robot by Rieffel et al. [12] are controlled with a spiking neural network. A genetic algorithm is used to optimize the weights of the network. The network architecture that exploits antagonist muscle actuation is hand-designed. By knowing the morphology and the desired behavior of the soft robot beforehand, the size of the optimization problem is reduced. However, when morphology and desired gait are unknown, these methods are infeasible. Spielberg et al. [13] present a method that simultaneously learns a latent state representation and a control policy. An auto-encoder is trained to reduce the state of a simulated soft robot, and a neural network is trained to select the best action given in a current state. This method is enabled by the use of a state-of-the-art soft robot simulator that is differentiable; see Ref. [14].

The work presented in Ref. [15] investigates the co-design of soft robots. For a fixed number of generations, they optimize morphologies with an L-system like generation method and a genetic algorithm. The designs have linear muscles that are placed randomly. During the evaluation phase, the muscles are simulated with an initial muscle firing pattern. After improving the muscle placement with this control constraint, the muscle placement is kept unaltered and the firing pattern is evolved. During this alternating optimization process, the morphology stays unaltered.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPlWarSqf_Zsos6Btdcv1TskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

The co-design of rigid robots is shown in Ref. [16]. Morphologies are generated using a graph grammar and the control for every generated design is found through model predictive control (MPC). MPC requires considerable computational power since every time-step the controls for a fixed horizon are optimized. Also, MPC is unlikely to find dynamic gaits [16].

3 Methods

This article presents a CDS method for the co-design of soft robot morphology and actuation. This section starts with a brief introduction of the methods proposed for morphology generation, evaluation, and optimization. The basic approach is the same as in the author's earlier work [5]; the designs are generated using a spatial grammar, evaluated in simulation and optimized using annealing.

Two methods for the optimization of the actuation of the designs are presented, followed by the integration of one of the methods into the CDS method to create a co-design method.

3.1 Computational Design Synthesis of Morphologies. The designs for the soft robot morphologies are generated from simple geometric building blocks; see Fig. 2(*a*). Spheres and cubes are unactuated materials; cylinders belong to assemblies that model soft bending actuators. The spatial grammar used for the design generation consists of four rules and is shown in Fig. 2(*b*).

The first rule adds a rigid building block, either a cube or a sphere, at a free connection point on the design. The four parameters of the rule are (1) the geometry that is added, (2) the part of the existing design to connect to, (3) the connection point of the existing part to connect to, and (4) the rotation of the newly added geometry. The second rule adds an actuator to the design. Parameters 1 and 2 in the rule are the part and connection points to connect to. The third parameter defines the length of the actuator in the number of cylinders and the fourth parameter assigns one of the six predefined actuation patterns to the actuator. The six patterns are shown in Fig. 3. The third rule modifies the rotation of a part or the actuation pattern of an actuator. The fourth rule revises a previous additive rule application, on the condition that the revision does not result in an invalid design, e.g., a design split in two. By mirroring each rule application in the *x*-*y* plane, a planar symmetry is achieved. This symmetry constraint is directly implemented in the grammar and holds for the entire generation process. When generating designs, the mirrored assembly has the phase-shifted activation of the original assembly. For example, if an actuator on the left side is activated with activation 1 from Fig. 3, then the right will be activated with activation 4.

The resulting soft robot designs are approximated with rigid-body dynamics simulation using the Bullet Physics Library [17]. The target values of the bending actuation of the actuators are achieved through the actuation of the hinge-constraints between cylinders. The objective value function to be maximized is given by Eq. (1) as follows:

$$f(\mathcal{D}) = \min_i |\mathbf{x}_i^{t_0} - \mathbf{x}_i^{t_{end}}| - \max\left(\frac{1}{N}\sum_{i=1}^{N} y_i^{t_0} - y_i^{t_{end}}, 0\right) \quad (1)$$
$$i = 1, \cdots, N$$

Here, $\mathcal{D}$ is the design, which consists of N elements. The two-dimensional vector $\mathbf{x}_i$ represents the x- and z-position of an element i. The time superscripts t_0 and t_{end} stand for the start and end of the simulation time. The y-position (height) of the element i is denoted as y_i. By selecting the least moving element of the design, a high objective function value (OFV) for designs that merely rotate is prevented. The second term of the equation is a penalty on the use of initial potential energy to cover the distance. Using SA, the morphologies are optimized. The SA optimization process is stopped when the defined convergence criterium is met, i.e., when no higher objective function value is found within 20 iterations.

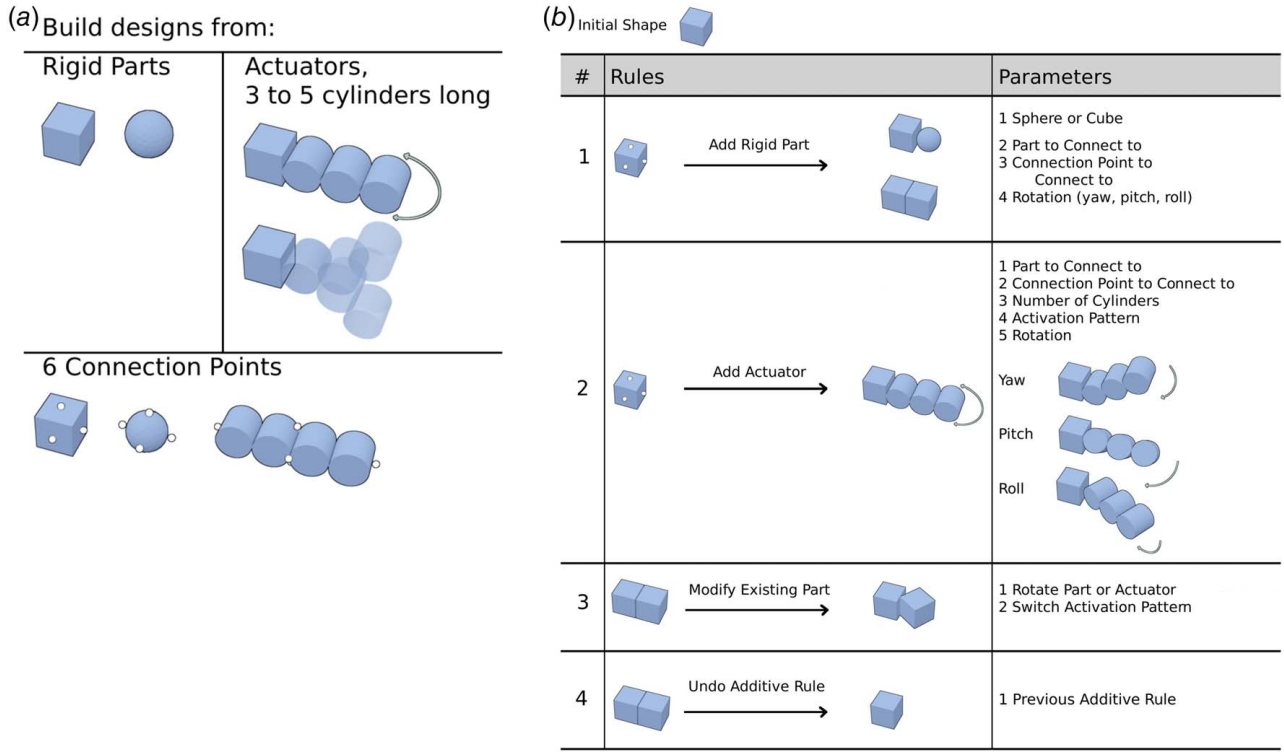


Fig. 2 (*a*) The building blocks of the spatial grammar. Spheres and cubes are single elements. Cylinders are part of bending sub-assemblies. Every building block has six defined locations to connect to other building blocks and (*b*) the spatial grammar rules used for the design generation.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPlJWarSqf_Zsos6Btdcv1TskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

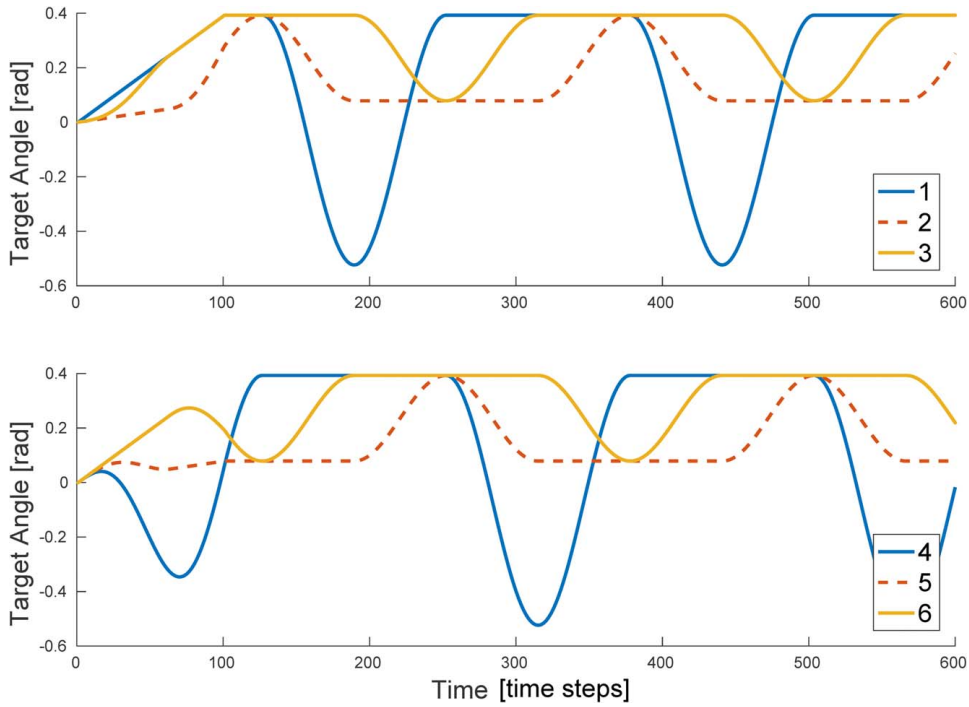


Fig. 3 Six predefined activation patterns. The bottom three are phase-shifted duplicates of the top three and are used to generate an alternating gait in symmetric designs by assigning phase-shifted patterns to the mirrored actuators.

3.2 Actuation Optimization. To test the suitability of approaches to actuation optimization, two methods for the generation of the actuation for one, generated soft robot morphology is presented. The first method, PAC optimization, models and optimizes feedforward control. The second method is SAC RL and generates a feedback controller. Based on the initial results (Secs. 4.1 and 4.2), only the implementation of PAC optimization is integrated into a CDS co-design process, which is presented in Sec. 3.3.

3.2.1 Parametric Actuation Curves Optimization. PACs describe the actuator target angles, similar to the predefined actuation patterns used in Sec. 3.1 and illustrated in Fig. 3. One PAC describes the angle of one actuator. For symmetric morphologies, each mirrored pair of actuators has an independent actuation curve. This assumption reduces the number of PACs to be optimized for symmetric designs by a factor of two.

PACs are formulated as Bezier curves; see Fig. 4. Bezier curves can describe a smooth actuation pattern based on a small set of variables. Each Bezier curve has five variables to optimize, four of which define the values of 15 curve points for an order 15 Bezier curve. The fifth variable defines the mapping of the Bezier curve to the timeline, thus determining the frequency. The start and end points of the curve are the same to enable smooth cyclic behavior. By forming sets of three identical curve points, the curves follow the curve points closer and the first and second derivatives at the start and end are matched. The control points of the curves are bound to the interval $[-2.5,2.5]$. The period of the Bezier curves is varied within the interval $[3.5,7.0]$. Every PAC has two additional binary variables. The first variable inverts the Bezier curve. For the actuation, this is equivalent to inverting the bending from positive to negative. The second variable defines if a mirrored actuator is actuated in phase or in counter-phase with the original actuator. This variable is only relevant for symmetric designs.

One actuation frequency per design is used to ensure that the gait simulated for 20 s is representative of the gait after this 20 s. When the actuation frequencies of the different actuators are not equal, the frequency of the combined actuation might be much longer than 20 s. With that assumption, the set of variables to optimize for the PAC actuation consists of four Bezier curve points, a binary inversion value, a binary mirror-phase value per PAC, and one frequency value for the set of PACs. The PACs are optimized with simulated annealing and evaluated using Eq. (2). Simulated annealing with a Cauchy cooling schedule [18] is used; see the pseudo code in Fig. 5.

The starting temperature T is 9.0 and is determined empirically, the cooling rate is 0.1. Reheating of 0.1 is used if the process has not accepted a new solution in the last 80 moves.

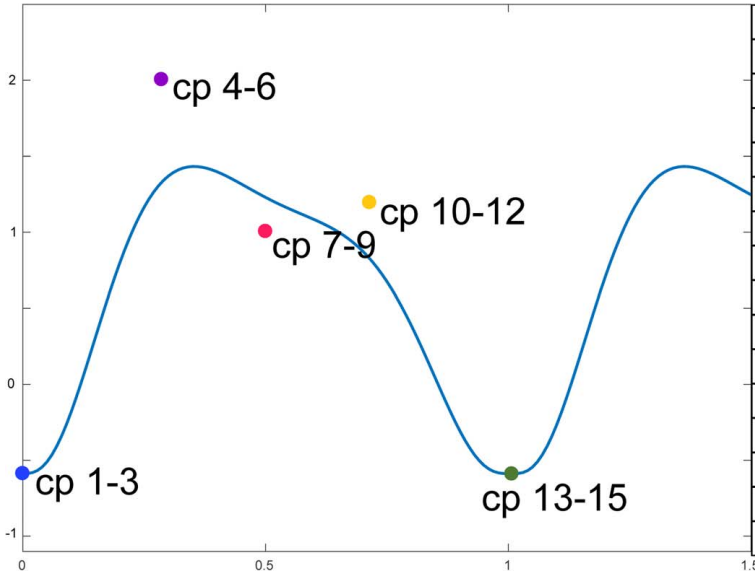
To generate a new PAC candidate, each Bezier curve point is perturbed with a probability of 0.5, and the period of the entire set of PACs is perturbed with a probability of 0.3. The size of the perturbation is defined by a normal distribution with $\mu = 0.0$ and $\sigma = \pi \times T$, where T is the current annealing temperature. The binary inversion and mirror-phase values for each PAC are inverted with a probability of 0.1.

3.2.2 Soft Actor-Critic Reinforcement Learning. Since the formulation of the PACs imposes limitations on the actuation, e.g., cyclic behavior, and they are a form of feedforward control, the actuation for several designs is learned using a state-of-the-art SAC RL method for comparison.

Here, a model-free RL method is selected since it is infeasible to construct a state transition-model with the used simulation model. SAC [19] is an off-policy actor-critic method with a continuous action space. SAC is a deep reinforcement learning method and uses neural networks (NN) to approximate the state-value and action-value functions in the actor and the critic. The variables subjected to learning are the weights of these networks. A schematic of actor-critic RL is shown in Fig. 6.

The environment shown in Fig. 6 is a soft robot in simulation. The action selected by the actor consists of a target angle increment for each actuator in the range $[-0.3,0.3]$ rad. To allow the selected action to have a measurable effect on the design, a new action is selected only three times per second while the simulation time-step is fixed at 0.01 s. From the environment, a state and a reward are fed back to the RL method. The state consists of four components as illustrated in Fig. 7. The first state component is the current actuator shape is determined by taking the average of the angles of the

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPjWarSqf_Zsos6Btdcv1TskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November



cp	x	y
1	0.00	-0.10
2	0.00	-0.10
3	0.00	-0.10
4	0.25	2.00
5	0.25	2.00
6	0.25	2.00
7	0.50	0.95
8	0.50	0.95
9	0.50	0.95
10	0.75	1.20
11	0.75	1.20
12	0.75	1.20
13	1.00	-0.10
14	1.00	-0.10
15	1.00	-0.10

Fig. 4 PAC: The left shows one and a half cycles of an order 15 Bezier curve and the right lists the *x* and *y* coordinates of the curve points. Every set of three Bezier curve points is aggregated and represented by a single variable for optimization. The last set is always equal to the first to ensure smooth cyclic behavior. By using an order 15 curve, the curves follow the curve points closer and the first and second derivatives at the start and end points are matched.

Algorithm 1 PAC Optimization with Simulated Annealing

Parameters: $\{T_0$, cooling rate, reheating value, nof interations before reheating, maximum nof iterations, nof moves$\}$
Initialize PAC $\mathcal{C}$ for morphology $\mathcal{D}$
for maximum nof iterations **do**
  for nof moves **do**
    Add v to the parameters of $\mathcal{C}$, with $v \to \mathcal{N}(\mu = 0.0, \sigma = \pi \cdot T)$
    Invert each PAC with probability 0.1 and modify the mirror-phase value for each PAC with probability 0.1
    Simulate $\mathcal{D}$ with PAC $\mathcal{C}$ for 20 seconds and determine the Objective Function Value (OFV)
    if OFV is best **then** store $\mathcal{C}_{best}$
    Depending on T, accept or reject $\mathcal{C}$
  if nof iterations no improvement > nof interations before reheating **then**
    $T = T +$ reheating value
  else
    Update T according to the Cauchy cooling schedule and cooling rate

Fig. 5 Pseudo code of simulated annealing for the optimization of PAC

hinges of the actuator. This approximation assumes a constant curvature over the actuator. The second component is the actuator speed which is measured as the difference between the current and previous actuator angle. Binary floor-contact sensors for each building block and three rotations for the center building block (initial shape as shown in Fig. 2(*b*)) are the last two components of the input.

A reward function defines the reward that is used to update the critic. Following the reward structure used for similar tasks by [19], the fixed dense reward function for the entire time horizon is given by:

$$r_t = W_{dist} \min_i \mathrm{x}_t^i + W_{velo} \dot{\mathbf{x}}_t^c \quad i = 1, \ldots, N \tag{2}$$

Here, r_t is the reward at control step t, W_{dist} and W_{velo} are the weight factors for the distance and velocity components, N is the number of elements in the design, x is the horizontal location of element i, and $\dot{\mathbf{x}}_t^c$ is the horizontal velocity of the center element, i.e., the initial shape depicted in Fig. 2, which is present in every design. The distance component of the reward depends on the shortest distance traversed by any element of the design, similar to the objective value function given by Eq. (2). The distance and the velocity weights are empirically chosen to be $W_{dist} = 2.0$ and $W_{velo} = 20.0$ by observing the results for different weights. It is observed that a lower relative velocity weight causes the process

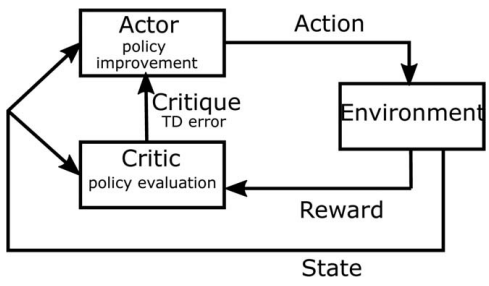


Fig. 6 A schematic of actor-critic RL

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPjWarSqf_Zsos6Btdcv1TskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

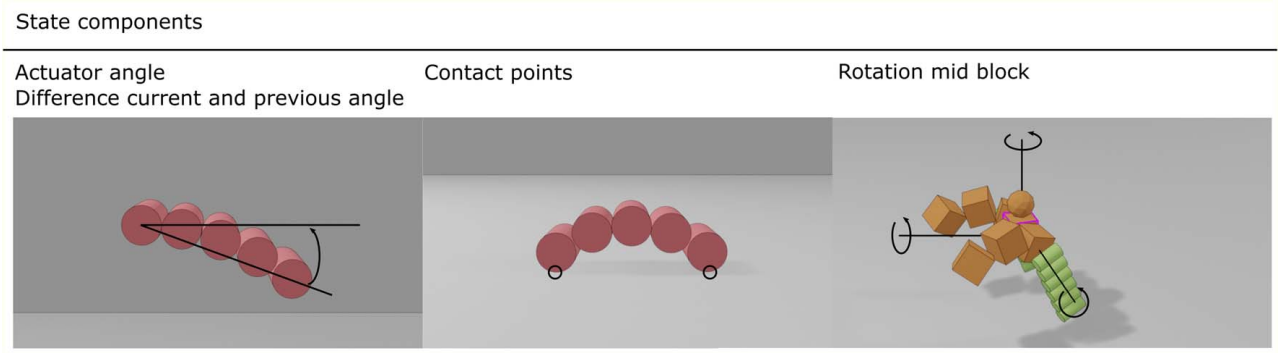


Fig. 7 The input state has four components: (*a*) the actuator angle, (*b*) the difference between the current angle and the angle at the previous time-step (left), and (*c*) the floor-contact points (middle) and three rotations for the center building block (right)

to settle in a local maximum where designs move appendices but not their core. A higher relative velocity weight rewards the designs that mainly use initial potential energy to move. SAC uses a modified reward function that consists of the reward itself, as described earlier, and a term to maximize the entropy of the policy. A policy with high entropy has small differences between the action selection probabilities, i.e., a flat Gaussian. Learning a maximum entropy policy increases exploration and a policy that is able to select good actions without unjust convergence on a single action makes the method less brittle [19]. The evaluation of the policy formulated in the critic is updated and the critic is used to improve the actor. With the new state as input, the improved actor selects the next action. The hyperparameters for SAC are chosen in accordance with the parameters used for the locomotion tasks in Ref. [19].

3.3 Co-Design Computational Design Synthesis Method Including Parametric Actuation Curves Optimization. PAC optimization is integrated into the CDS method to optimize the actuation for each generated morphology; see the schematic in Fig. 1. The process starts by generating a morphology using the presented spatial grammar (Fig. 2). In the actuation optimization loop, or inner loop, a set of PACs is optimized as follows. The set of PACs is

Algorithm 2 Computational Design Synthesis including PAC Optimization

Parameters morphology optimization loop: $\{T_0$, cooling rate, maximum nof iterations, nof moves, maximum nof parts, maximum nof actuators, max fraction of actuated / passive $\}$
Parameters PAC optimization loop: $\{T_0^c$, cooling rate^c, maximum nof iterations^c, nof moves$^c\}$
Convergence criteria for the morphology loop: { max iterations without improvement}
Convergence criteria for the PAC loop: { fraction (improvement / best value) during last ϕ iterations $< Frac_{convergence}\}$
Initialize Design $\mathcal{D}$
for maximum nof iterations **do**
  for nof moves **do**
    if maximum nof parts not reached **then**
      if maximum nof actuators not reached AND max fraction of actuated / passive not reached **then**
        apply 1 spatial grammar rule { add / delete / modify } randomly to $\mathcal{D}$
      else
        apply 1 spatial grammar rule { add passive part / delete / modify } randomly to $\mathcal{D}$
    else
      apply 1 spatial grammar rule { delete / modify } randomly to $\mathcal{D}$
    Initialize PAC $\mathcal{C}$
    for maximum nof iterations^c **do**
      for nof moves^c **do**
        Add v to the parameters of $\mathcal{C}$, with $v \rightarrow \mathcal{N}(\mu = 0.0, \sigma = \pi \cdot T^c)$
        Invert each PAC with probability 0.1 and modify the mirror-phase value for each PAC with probability 0.1
        Simulate $\mathcal{D}$ with PAC $\mathcal{C}$ for 20 seconds and determine the Objective Function Value (OFV)
        if OFV is best **then** store $\mathcal{C}_{best}$
        Depending on T^c, accept or reject $\mathcal{C}$
        if convergence criteria met **then** break
      Update T^c according to the Cauchy cooling schedule and cooling rate^c
    Set OFV of $\mathcal{C}_{best}$ as OFV for morphology and actuation design $\mathcal{CD}$
    if OFV is best **then** store $\mathcal{CD}_{best}$
    Depending on T, accept or reject $\mathcal{CD}$
  if convergence criteria met **then** break
  Update T according to the Cauchy cooling schedule and cooling rate

Fig. 8 Pseudo code for the CDS method with integrated PAC optimization

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPlWarSqf_Zsos6Btdcv1TskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

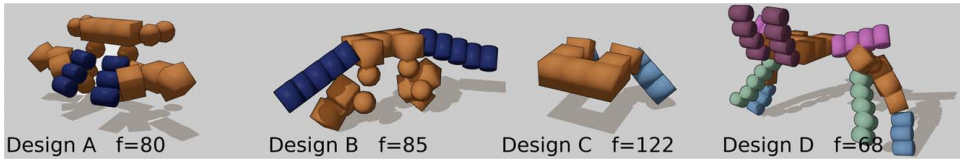


Fig. 9 Designs A–D result from the generation with predefined actuation patterns. Designs A, B, and C have two actuators, and Design D has eight actuators. The OFVs *f* after 20 s of simulation with predefined actuation patterns are shown. Balls and cubes are unactuated, stiff materials. The rows of cylinders forming actuators are shown in different shades. Actuators that are symmetric counterparts are shown in the same shade.

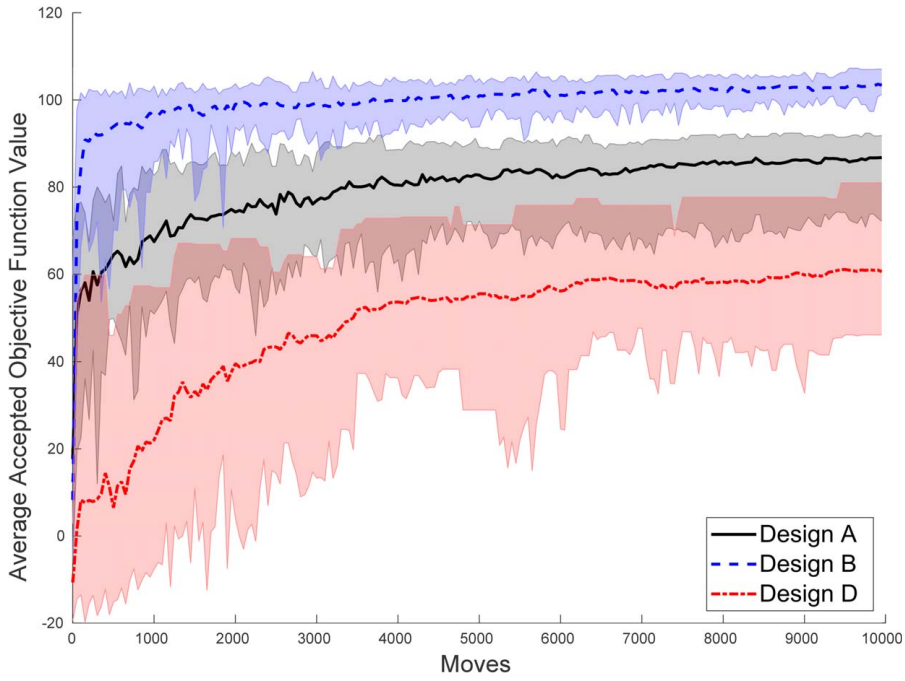


Fig. 10 Average accepted objective function value and the range versus moves for PAC optimization. A PAC is optimized for Designs A, B, and D. For each design, the average accepted objective function value and the range of 22 runs are shown.

Table 1 The PAC optimization results compared to the predefined actuation result for three designs

Design	Predefined actuation result	Best result/predefined actuation result	Worst result/predefined actuation result	Runs better than predefined/all runs
Design A	80.0	1.16	0.99	19/22
Design B	85.4	1.26	1.22	22/22
Design D	67.6	1.20	0.73	9/22

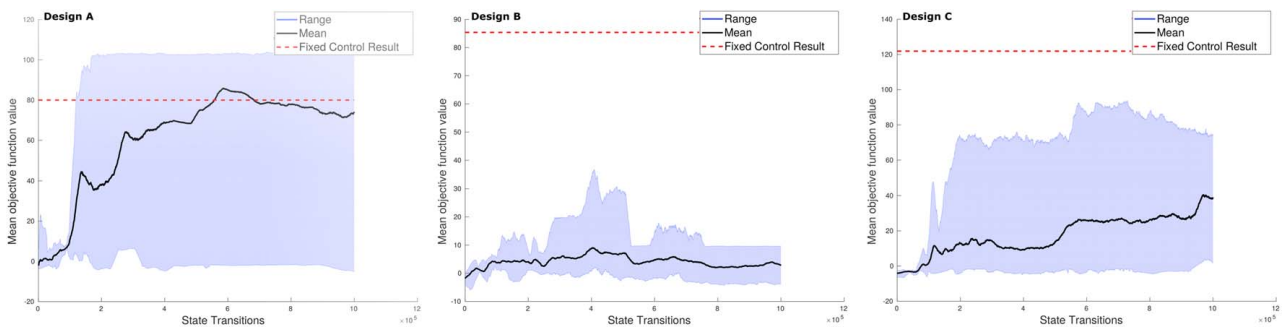


Fig. 11 Average objective function value plotted against state transitions for nine SAC runs for Designs A, B, and C. For each design, the objective function value achieved with predefined actuation is plotted as a dashed line.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPlWarSqf_Zsos6Btdcv1TskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

Table 2 The best-found SAC results compared to the predefined actuation result for three designs

Design	Predefined actuation result	Best result/predefined actuation result	Worst result/predefined actuation result	Runs better than predefined/ all runs
Design A	80.0	1.37	0.99	8/9
Design B	85.4	044	0.28	0/9
Design C	121.9	0.77	0.35	0/9

initialized, the combination of morphology and PACs is evaluated in simulation, the set of PACs is accepted or rejected by simulated annealing, and the PAC variables are modified to generate the set for the next iteration. The actuation optimization loop is terminated when convergence criteria are met, or when a maximum number of iterations is reached. The objective function value of the best PAC is used as the objective function value of the combination of morphology and actuation. In the outer loop, the combination of morphology and actuation is accepted or rejected by the simulated annealing method. The next iteration starts with generating a modified morphology by applying a spatial grammar rule to the current design. To clarify, the pseudo code of the co-design method is shown in Fig. 8.

4 Results

In this section, the results of the experiments with the different methods are presented. First, several results of the design of morphologies with predefined actuation are presented. These designs, generated by the CDS method for morphologies only, are used to examine the performance and usability of the two actuation optimization methods proposed in Sec. 3 and are presented in Secs. 4.1 and 4.2. Finally, the results of a CDS method for the co-design of morphology and actuation are presented in Sec. 4.3. The result section is concluded in Sec. 4.4 with a case study of replicating the generation of an existing soft robot to show that it exists within the design space of the CDS method.

For the comparison of the PAC and SAC actuation optimization methods, four designs generated with the CDS method for generating morphologies with predefined actuation are shown in Fig. 9. For example, Design A is a result with two actuators. After a simulation period of 20 s, Design A has an objective function value, f, of 80. As can be seen from Designs B, C, and D, the spatial grammar enables the generation of distinctive designs with a varying number of actuators. Three of the designs are used in each study of PAC and SAC.

4.1 Parametric Actuation Curves Results. First, PAC optimization is applied to three of the designs as shown in Fig. 9. Actuation is optimized for Designs A and B with two actuators, and Design D with eight actuators; see Fig. 9. A successful gait for Design C has to be dynamic and is therefore highly sensitive and too sensitive for PAC optimization. Design C is therefore excluded here. The simulated annealing process runs 200 iterations, with 50 moves per iteration. The average accepted objective function value and range versus the number of moves of 22 runs of PAC optimization for the three different designs are shown in Fig. 10. In Table 1, the PAC optimization results are compared to the performance of the predefined actuation results. Although not every run finds a better actuation, all runs find actuation patterns that reach at least 70% of the objective function value of predefined actuation,

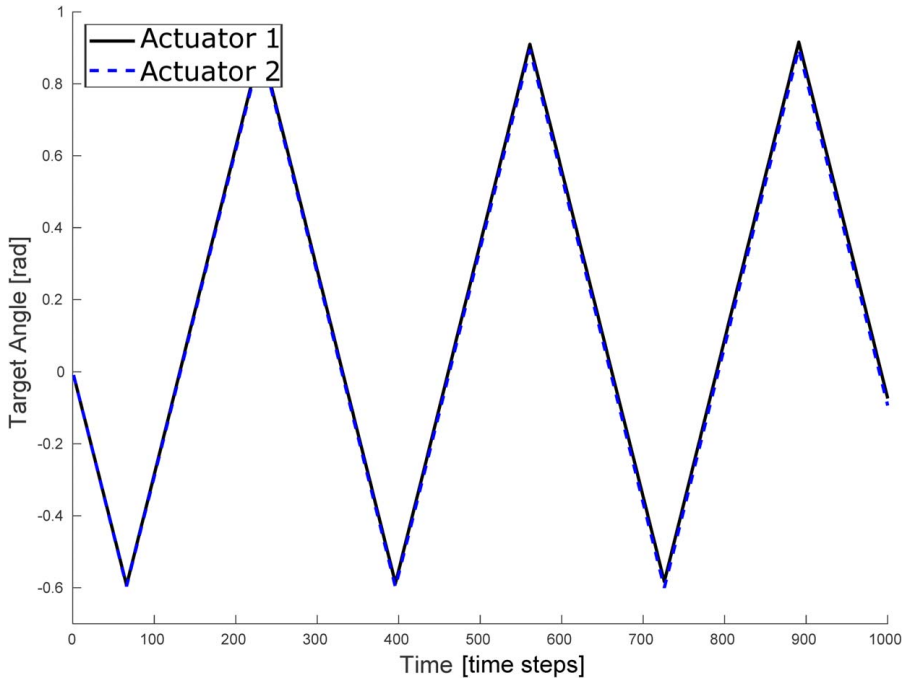


Fig. 12 Target angles for the two actuators of Design A are given by the policy learned with SAC. As shown, the actions of the two actuators only differ slightly and the control is cyclic. Note that the actual actuator angles in simulation will show a smoother progression due to the inertia of the morphology.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPlWarSqf_Zsos6BtdcvITskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

Table 3 Parameters used for the co-design CDS

Morphology optimization outer loop)		Actuation optimization (inner loop)	
Annealing temperature	9.0	Annealing temperature	9.0
Annealing cooling rate	0.1	Annealing cooling rate	0.1
Maximum # of iterations	100	Maximum # of iterations	300
# of moves	3	# of moves	5 for actuators < 3
Maximum # of parts	20		10 for 2 < actuators < 5
Maximum # of actuators	8		15 for 4 < actuators
Maximum # of iterations with no improvement	5		
# of iterations of no improved objective function for convergence	20		

even for designs with eight actuators. For each design, multiple actuation patterns are found that outperform the predefined actuation.

4.2 Soft Actor-Critic Results. The SAC implementation by Haarnoja [20] is used to learn a controller for Design A from Fig. 9. When adding SAC to the CDS method as shown in Fig. 1, the morphology considered changes in every design iteration. As a consequence, SAC either must use a single set of parameters for all morphologies, or it has to be feasible to auto-tune the learning parameters and reward function. To assess the usability of SAC as the controller method within the co-design loop, the same SAC parameters are used to learn controllers for Designs B and C. Since Designs B and C have the same number of actuators as Design A, the learning parameters and reward function used for Design A are expected to work for these designs as well. Design D is excluded since the high number of actuators result in a large action space, which is computationally heavy to explore. In addition, the reward function used for the designs with two actuators does not fit for a design with eight actuators.

Running nine SAC experiments, each with 10^5 state transitions, for Design A results in a best-learned policy with an objective function value of 109.4, which is around 37% better than the predefined actuation. Figure 11 (left plot) shows the average objective function value and its range plotted against the number of state transitions. The range of the objective function values of the policies is very large, which means that not every SAC run converges to a solution that is better than the predefined actuation result. The parameters used were the best found among 150 combinations tried. However, better convergence might be achieved by further tuning of the reward function and learning parameters. Even though

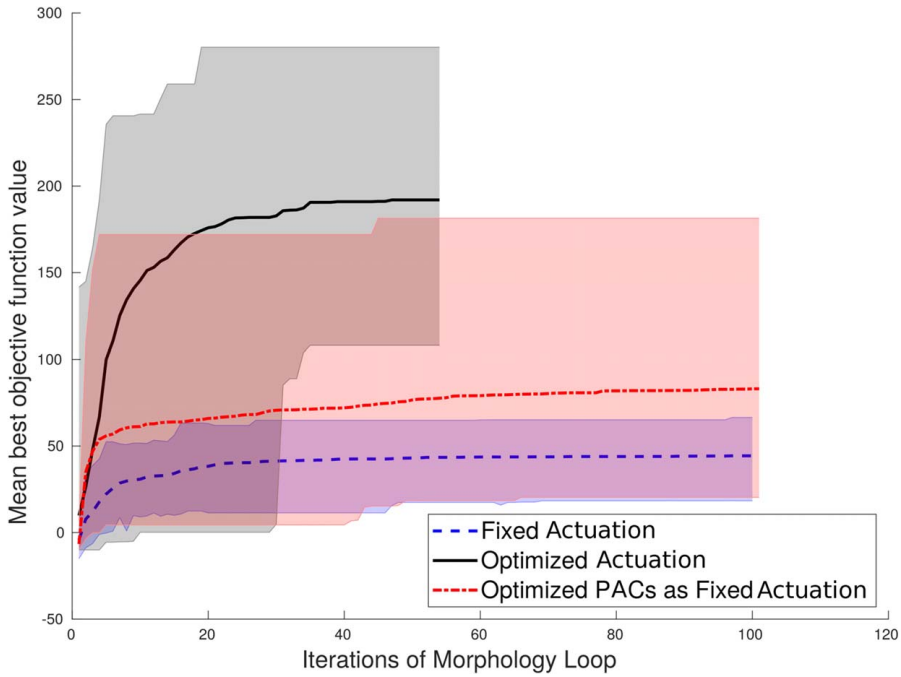


Fig. 13 The average and range of the best objective function value versus the iterations is plotted to compare (1) the design of morphology only (fixed actuation), (2) the co-design of morphology and actuation (optimized actuation), and (3) the design of morphology with fixed PACs as predefined actuation (optimized PACs as fixed actuation). The design process of morphology only is run 30 times and has 100 iterations and 25 moves per run. The design process with PAC optimization is run 30 times and each run has a maximum of 100 outer loop iterations with every three moves and a maximum of 300 inner loop iterations with every five moves. The process with actuation optimization is terminated when the convergence criteria are met. The actuation patterns found with co-design (solid line) perform worse when these exact patterns are used as fixed actuation patterns in CDS (dash-dot line), i.e., morphologies fitting these patterns are sought. Therefore, the difference between the CDS with fixed actuation (dashed line) and the co-design (solid line) cannot be explained by a bad choice of fixed actuation patterns only and must also be due to the advantage of co-design.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPlWarSqf_Zsos6BtdcvITskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

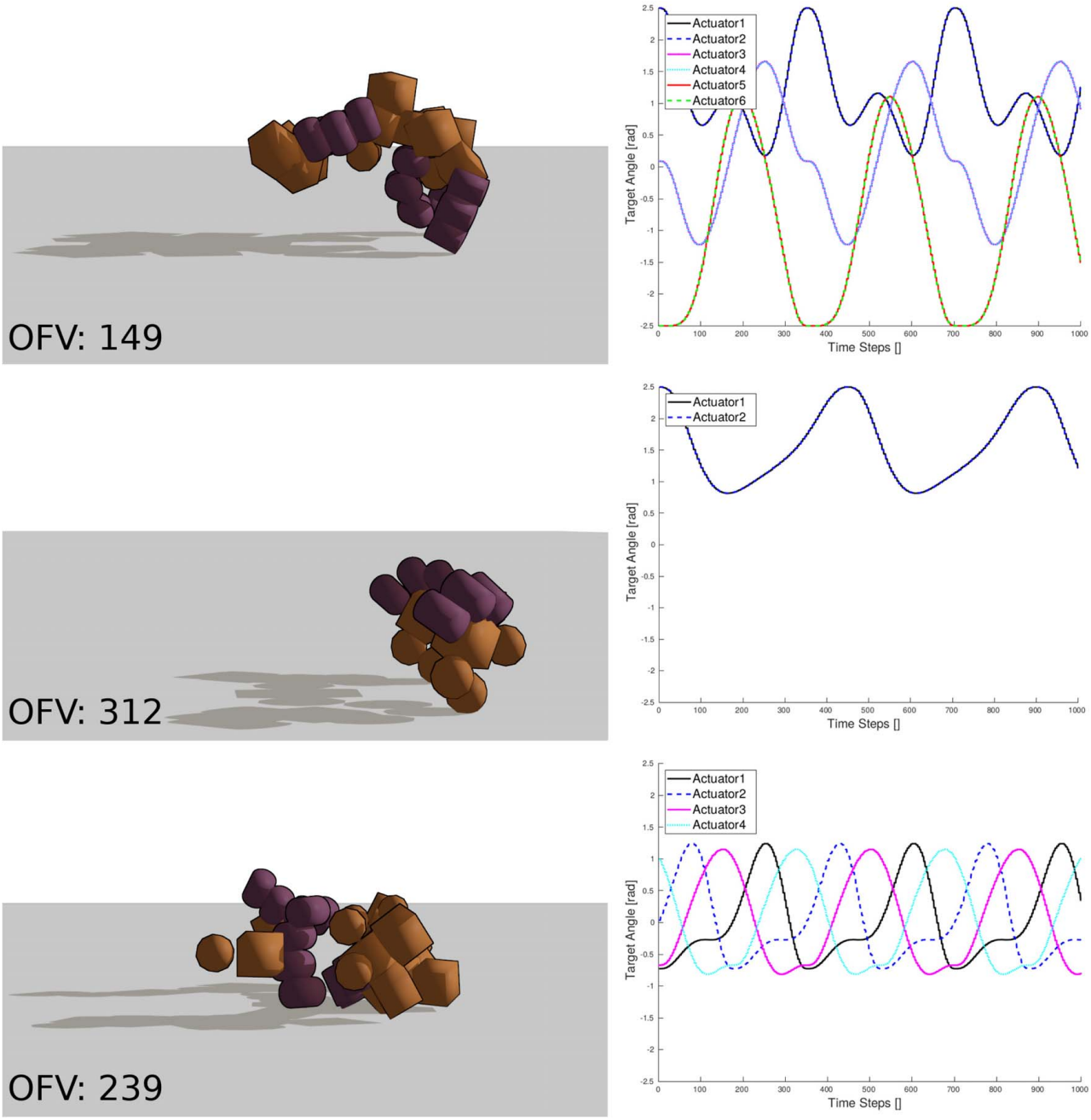


Fig. 14 Three results from the co-design of morphology and control using PAC are shown. The top design has six actuators, where the symmetric actuators are actuated in phase. It uses two sets of actuators to produce a hopping motion and an extra set to keep its balance. The middle design rolls. The bottom design has four symmetric actuators in the counter-phase. One pair of actuators releases the pressure on one side of the body, the other pair pushes off. For each design, the OFV is obtained.

convergence is desirable, storing the best found controller is sufficient in this case.

As can be seen from the middle and right plots in Fig. 11, using the parameters tuned for Design A for the learning processes of Designs B and C does not result in good controllers. None of the 18 runs reported the best objective function value that was better than the predefined actuation objective function values, meaning that the used parameters and reward function do not fit other designs with the same degree-of-freedom as Design A. The results of the SAC experiments are summarized in Table 2. Figure 12 shows the learned policy for Design A.

4.3 Results of the Co-Design of Morphology and Actuation. PAC is implemented to create a co-design CDS method that optimizes the actuation patterns within the CDS. PAC is chosen based on the superior results shown in Table 1 versus Table 2, and since further investigation is needed into the tuning of SAC for each new morphology generated and into the possibility of using hyperparameters. The results in Sec. 4.2 show that PAC optimization results in acceptable performance for the designs considered, with the exception of Design C. The parameters for the co-design CDS method shown in Fig. 8 are listed in Table 3 and are determined empirically through optimizing the actuation patterns for Designs A, B, and D. The parameters are a trade-off between computational cost and performance. The parameters for the outer loop, i.e., morphology and actuation combined, follow from the experiments with the design of morphology only. As shown in Table 3, the number of moves per actuation

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPjWarSqf_Zsos6BtdcvITskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

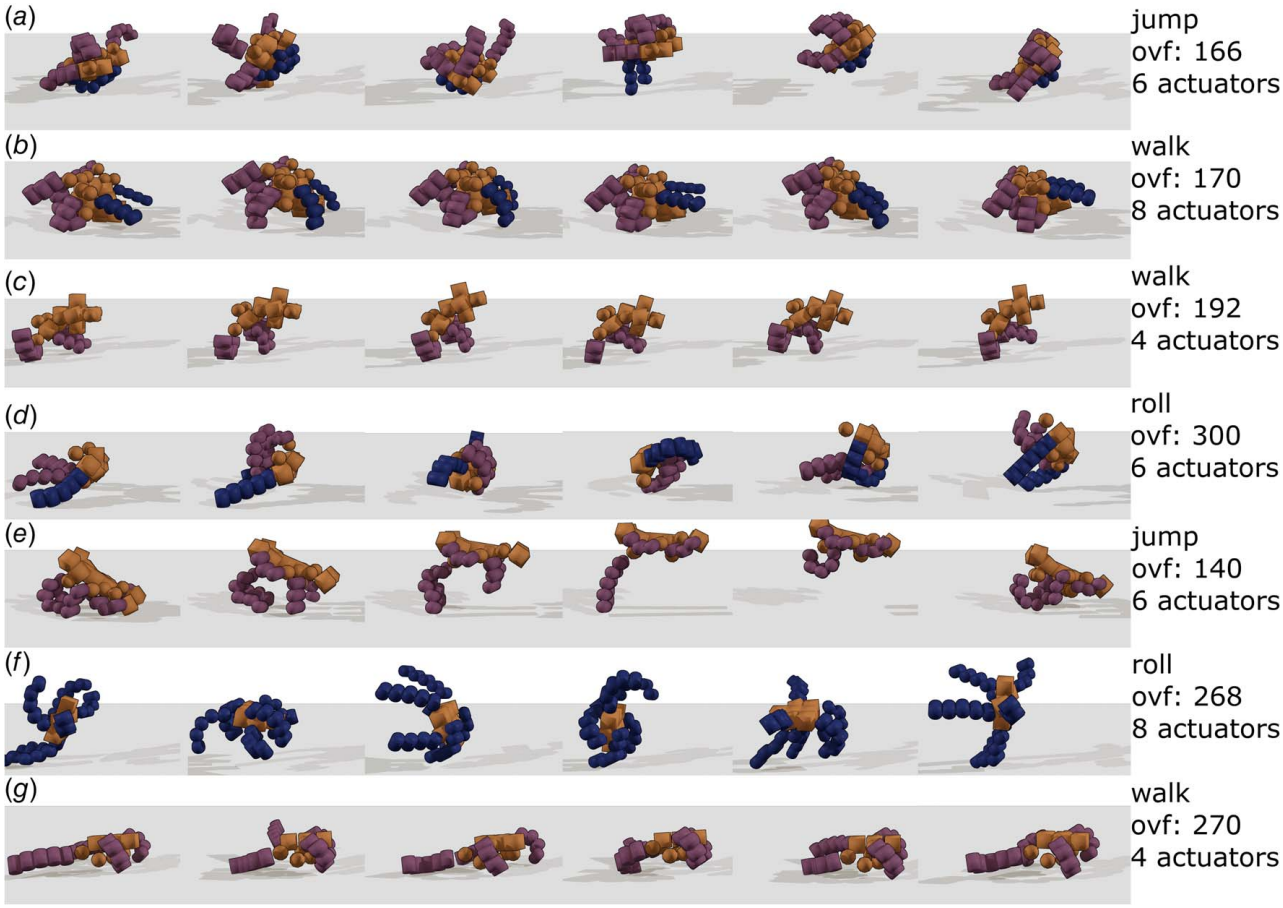


Fig. 15 Seven designs resulting from the co-design of morphology and control using PAC. The images show the design moving from left to right. The type of gait (walk, roll, and jump), the OFV, and the number of actuators are stated on the right for each design.

optimization iteration depends on the number of actuators of the design.

The co-design of morphology and actuation is compared to the design of morphology only; see Fig. 13. The figure shows the convergence for three types of CDS processes: (1) CDS with predefined actuation patterns as described in Sec. 3.1, (2) co-design CDS with PACs, and (3) CDS of morphology only with PACs as predefined actuation patterns. Comparing processes 1 and 2, it can be seen that integrating the co-design of the actuation patterns improves the best objective function value by around 350% and needs fewer morphology iterations.

However, to make a fair comparison, a third experiment is required. Comparing the performance of a CDS with optimized actuation optimization to a CDS without actuation optimization is not a fair comparison for two reasons. First, the energy available for a design is not equal. Due to the optimization of the actuation frequency and the variable amplitude of the PACs, the amount of energy a design uses is modified. Thus, designs actuated with optimized PACs potentially use more energy than designs with predefined actuation and that could be a reason for their better performance. Second, it is difficult to determine if the six predefined actuation patterns were selected well.

To make a fair comparison between the CDS method with and the CDS method without actuation pattern optimization, the 30 PACs found in the 30 co-design runs conducted in process 2 are used as predefined actuation in a third design process for the generation of design morphologies only. Thus, it is known that successful morphologies exist for these sets of PACs.

As shown in Fig. 13, the design of morphology with optimized PACs as predefined actuation patterns (process 3) confirms that these PACs allow for the synthesis of better solutions compared to the hand-designed actuation patterns from Fig. 3 that are used in process 1. However, comparing processes 2 and 3 shows that finding a morphology to match a fixed set of PACs is harder and less successful than co-designing a morphology and a set of PACs simultaneously.

To illustrate the variety of resulting actuation patterns from the CDS co-design method, Fig. 14 shows three morphologies and corresponding actuation. Further, a diverse collection of designs that exhibit walking, rolling, and jumping gaits is presented in Fig. 15, underlining the vast solution space of the co-design CDS method for morphology and actuation presented here.

4.4 Results of Replicating an Existing Soft Robot. An existing state-of-the-art soft robot is replicated to show that the design is part of the solution space spanned by the proposed CDS of morphology and actuation. The robot selected is the multi-gait soft robot presented by Shepherd et al. [21] and it results from research dedicated to building a soft robot for locomotion. The robot exhibits an undulating and a crawling gait. A video of the design can be found.[2] Reverse engineering the multi-gait soft robot will answer three questions; (1) Can the generation method generate this design?, (2) Does the actuation pattern optimization find the same two gaits?, and (3) Is the simulated performance comparable to the reported performance?

First, the morphology is replicated by hand-picking six rules from the spatial grammar; see Fig. 16(*a*). The dimensions of the

[2]https://youtu.be/2DsbS9cMOAE

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPlWarSqf_Zsos6BtdcvITskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

rules are modified to model the actuators used for this robot. Thus, it is ensured that the solution space spanned by the proposed spatial grammar includes this design. As shown in Fig. 16(*a*), the physical robot has an extra actuator in the spine, reportedly to lift the center of mass and overcome friction. This actuator would have four connection points and is not possible within the used grammar rules. However, this extra actuator is needed because the molded silicone has low stiffness when unactuated and is considered a part of detail design, not conceptual design.

Second, the weights and forces reported in the article are used to adjust the simulation model and PAC optimization is used to find an actuation pattern for the design. Both the undulating gait and alternating gait executed by the physical robot are found as possible gaits by the PAC optimization process; see Fig. 16(*b*). Shepherd et al. report velocities for both an undulating gait (13 m/h) and a crawling gait (24 m/h), expressed in body lengths per hour, i.e., 93 and 192 body lengths per hour. The conceptual design in simulation reaches 150 body lengths per hour with an undulating gait and 291 with a crawling gait. Thus, Shepherd's multi-gait robot attains 62% and 65% of the simulated speeds. Since the actuation in the simulation is described by a target angle and the constraints cannot be expressed in forces directly, no comparison of the actuation force or pressure can be made. Although the actual speed of the robot is lower than that of the simulated robot, for both gaits the actual speed is around 60% of the simulated speed. This suggests that the simulation systematically overestimates the design, however, that makes the actual performance of an optimized third gait predictable.

5 Discussion

The proposed PAC optimization method improves the performance of the designs generated by the CDS method for morphology only. The parameters used for the optimization of the PACs are valid for all designs tested, without modification or re-tuning. The exception is the number of simulated annealing moves for the optimization of the PACs, which is dependent on the number of actuators and is adjusted automatically. The domain specific knowledge encoded in the PAC formulation, i.e., cyclic actuation and symmetry assumptions, restricts the solution space but enables the optimization of actuation patterns for up to eight degrees-of-freedom. From Fig. 12, it is seen that the policy learned with SAC for Design A results in cyclic and symmetric behavior, although no such assumptions are made. This suggests that the assumption of cyclic and symmetric actuation patterns for the formulation of PACs does not exclude the best solutions.

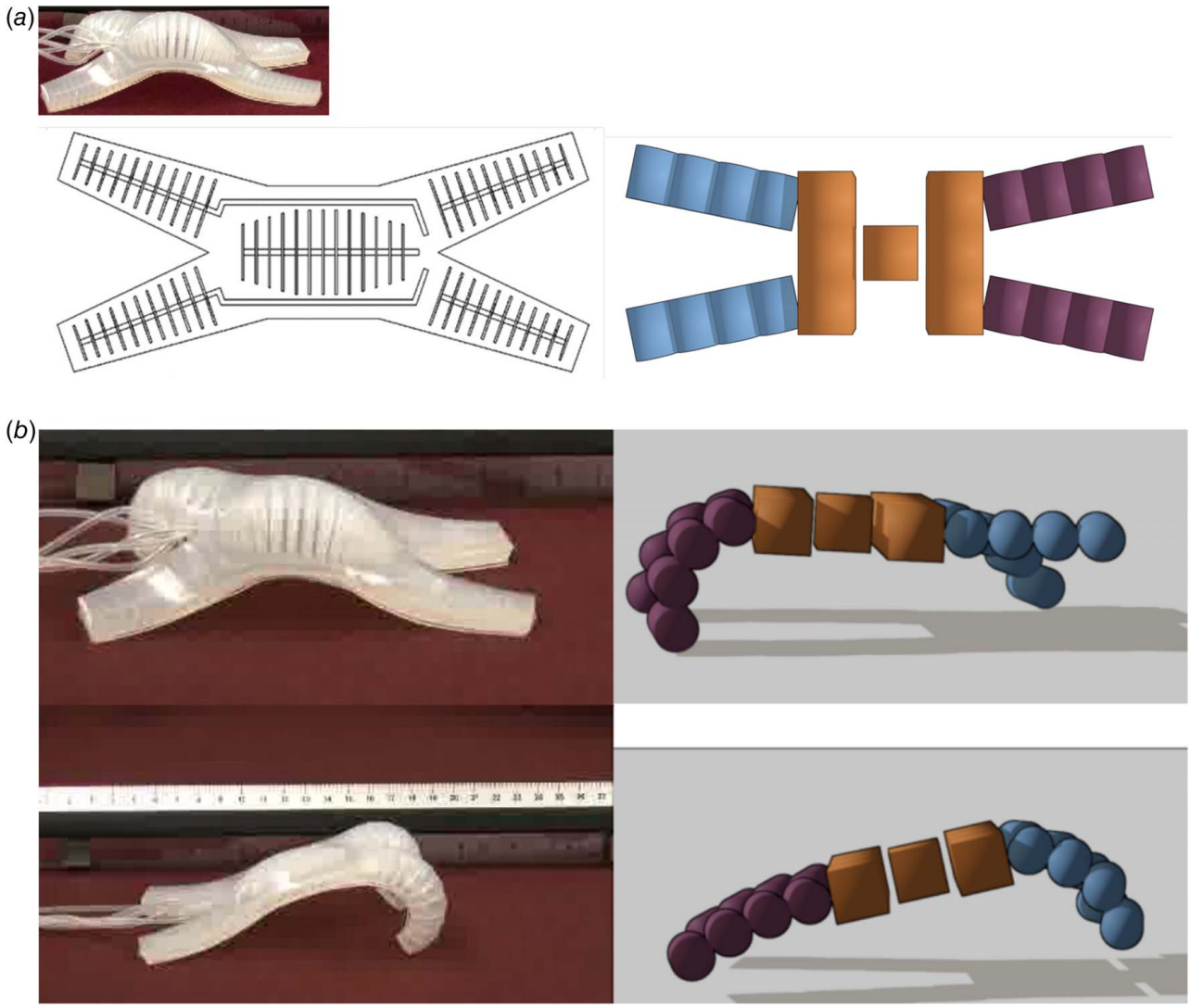


Fig. 16 (*a*) The hand-designed physical soft robot presented by Shepherd et al. and the same design generated by hand-picking spatial grammar rules and (*b*) The alternating gait (top) and the undulating gait (bottom) of the physical robot from [21] and the generated equivalent

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=weJtrWw4uQYAAAAA:SAfYCjq32Vp15yq9qlCmGqWPlWarSqf_Zsos6Btdcv1TskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November

However, for more complex tasks, it will be increasingly difficult to formulate valid assumptions to base the PAC formulation on, e.g., when a task is non-cyclic. When the number of actuators increases, the actuation pattern optimization problem scales linearly. The possibility to formulate dependencies between the actuation patterns for different actuators can reduce the size of the optimization problem.

The experiments with SAC result in controllers with good performance, i.e., high objective function values. The resulting feedback controller can be used as a controller for a physical robot and allows for improvement and tuning onboard. It is also a good base for learning more complex tasks, e.g., locomotion on rough terrain or locomotion with a sense of direction. However, it is deemed intractable to use SAC as part of a CDS method since the reward function and the learning parameters require design specific tuning. Since SAC has a relatively small number of tuning parameters for an RL method, a form of auto-tuning might be conceivable, however, this needs investigation. An additional interesting research direction is to start an SAC optimization with some form of pre-knowledge, e.g., the "seeding" of SAC with PAC results.

The results of the co-design of morphology and actuation show that co-design outperforms the design of morphology in two aspects. First, fewer morphological design iterations are needed, and second, the final performance is better. Additionally, the solution space is expanded since the predefined actuation patterns can be formulated as PACs and therefore, the set of all designs with predefined actuation is a subset of all designs with optimized actuation. The design of morphology only with optimized PACs as predefined actuation patterns confirms that these PACs allow for the synthesis of better solutions than the hand-designed actuation patterns from Fig. 3. However, Fig. 13 also shows that co-designing the morphology and actuation is more effective and efficient than designing a morphology to fit a predefined actuation pattern.

The used simulation method simulates soft actuator behavior, not the soft material itself, and assumes rigid point connections between body parts. To verify the validity of these assumptions, the designs resulting from the co-design process are prototyped in additional work by van Diepen [22]. In addition, their speed and novelty are compared to a set of existing soft robots.

6 Conclusion

This article presents a CDS method for the automated co-design of morphology and actuation of soft robots. Two methods for the optimization of actuation are considered; PAC optimization and SAC reinforcement learning. PAC assumes a cyclic gait and models actuation patterns for each actuator. SAC removes this assumption as well as modeling feedback control for comparison. Adding PAC optimization to the CDS method to create a co-design CDS method is shown to improve the performance of the resulting designs and to achieve better results within fewer design iterations. SAC is, however, deemed ineffective, due to the need for design specific problem tuning for each new morphology. The SAC experiments also show that the best found soft robot gaits are symmetric and cyclic, although this is not a constraint in the SAC problem formulation, thus verifying the assumptions made in the PAC formulation. Ten different designs automatically generated using the co-design CDS method illustrate the diverse search space explored and resulting solutions that demonstrate walking, rolling, and jumping gaits. To validate the resulting co-design method, a state-of-the-art soft robot is replicated and shown to be part of the solution space spanned by the CDS method.

Conflict of Interest

There are no conflicts of interest.

References

[1] Haddadin, S., Albu-Schäffer, A., and Hirzinger, G., 2009, "Requirements for Safe Robots: Measurements, Analysis and New Insights," Int. J. Robot. Res., **28**(11–12), pp. 1507–1527.

[2] Brown, E., Rodenberg, N., Amend, J., Mozeika, A., Steltz, E., Zakin, M. R., Lipson, H., and Jaeger, H. M., 2010, "Universal Robotic Gripper Based on the Jamming of Granular Material," Proc. Natl. Acad. Sci. U. S. A., **107**(44), pp. 18809–18814.

[3] Deimel, R., and Brock, O., 2013, "A Compliant Hand Based on a Novel Pneumatic Actuator," Proceedings—IEEE International Conference on Robotics and Automation, Karlsruhe, Germany, May 6–10, pp. 2047–2053.

[4] Seibel, A., and Schiller, L., 2018, "Systematic Engineering Design Helps Creating New Soft Machines," Robot. Biomim., **5**(5), pp. 1–10.

[5] van Diepen, M., and Shea, K., 2019, "A Spatial Grammar Method for the Computational Design Synthesis of Virtual Soft Locomotion Robots," ASME J. Mech. Des., **141**(10), p. 101402.

[6] Cheney, N., MacCurdy, R., Clune, J., and Lipson, H., 2013, "Unshackling Evolution: Evolving Soft Robots With Multiple Materials and a Powerful Generative Encoding," Proceeding of the Fifteenth Annual Conference on Genetic and Evolutionary Computation—GECCO'13, Amsterdam, The Netherlands, July 6–10, p. 167.

[7] Sims, K., 1994, "Evolving Virtual Creatures," Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1994, SIGGRAPH, Orlando, FL, July 24–29, pp. 15–22.

[8] Lipson, H., and Pollack, J. B., 2000, "Automatic Design and Manufacture of Robotic Lifeforms," Nature, **406**(August), pp. 974–978.

[9] Lund, H., Hallam, J., and Lee, W.-P. L. W.-P., 1997, "Evolving Robot Morphology," Proceedings of 1997 IEEE International Conference on Evolutionary Computation (ICEC '97)., Indianapolis, IN, Apr. 13–16.

[10] Stump, G. M., Miller, S. W., Yukish, M. A., Simpson, T. W., and Tucker, C., 2019, "Spatial Grammar-Based Recurrent Neural Network for Design Form and Behavior Optimization," ASME J. Mech. Des., **141**(12), p. 124501.

[11] Ishige, M., Umedachi, T., Taniguchi, T., and Kawahara, Y., 2019, "Exploring Behaviors of Caterpillar-Like Soft Robots With a Central Pattern Generator-Based Controller and Reinforcement Learning," Soft Robot., **6**(5), pp. 579–594.

[12] Rieffel, J., Rife, J., Trimmer, B., and Saunders, F., 2009, "Evolving Soft Robotic Locomotion in PhysX," Proceedings of the 11th Annual Conference Companion on Genetic and Evolutionary Computation Conference: Late Breaking Papers, Montreal, QC, Canada, July 8–12, pp. 2499–2504.

[13] Andrew Spielberg, W. M., Zhao, A., Du, T., Hu, Y., and Rus, D., 2019, "Learning-In-The-Loop Optimization: End-to-End Control and Co-Design of Soft Robots Through Learned Deep Latent Representations," Conference on Neural Information Processing Systems (NeurIPS), Vancouver, Canada, Dec. 8–14, pp. 8284–8294.

[14] Hu, Y., Liu, J., Spielberg, A., Tenenbaum, J. B., Freeman, W. T., Wu, J., Rus, D., and Matusik, W., 2019, "Chain-Queen: A Real-Time Differentiable Physical Simulator for Soft Robotics," Proceedings—IEEE International Conference on Robotics and Automation, Montreal, QC, Canada, May 20–24, pp. 6265–6271.

[15] Rieffel, J., Knox, D., Smith, S., and Trimmer, B., 2013, "Growing and Evolving Soft Robots," Artif. Life, **20**(1), pp. 143–162.

[16] Zhao, A., Xu, J., Konakovíc-Lukovíc, M., Hughes, J., Spielberg, A., Rus, D., and Matusik, W., 2020, "Robogrammar: Graph Grammar for Terrain-Optimized Robot Design," ACM Trans. Graph., **39**(6), pp. 1–16.

[17] Couman, E., 2015, "Bullet Physics Simulation." ACM SIGGRAPH 2015 Courses, p. 1.

[18] Copas, J., 1984, "On the Unimodality of the Likelihood Function for the Cauchy Distribution," Biometrika, **62**(3), pp. 701–704.

[19] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S., 2018, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," International Conference on Machine Learning, Stockholm, Sweden, July 10–15.

[20] Haarnoja, T., 2018, Github Soft Actor-Critic, https://github.com/haarnoja/sac.

[21] Shepherd, R. F., Ilievski, F., Choi, W., Morin, S. A., Stokes, A. A., Mazzeo, A. D., Chen, X., Wang, M., and Whitesides, G. M., 2011. "Multigait Soft Robot," Proc. Natl. Acad. Sci. U. S. A., 108(51), pp. 20400–20403.

[22] van Diepen, M., 2020, "Computational Design Synthesis for the Co-Design of Morphology and Actuation of Soft Robots for Locomotion," PhD thesis, ETH Zurich, Switzerland.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/144/8/083305/6886716/md_144_8_083305.pdf?casa_token=we.trWw4uQYAAAAA:SAYCjq32Vp15yq9qlCmGqWPjWarSqf_Zsos6BtdcvITskfCWQXw_mNK6UaSRrsLBYeg by Universita Di Pisa user on 27 November